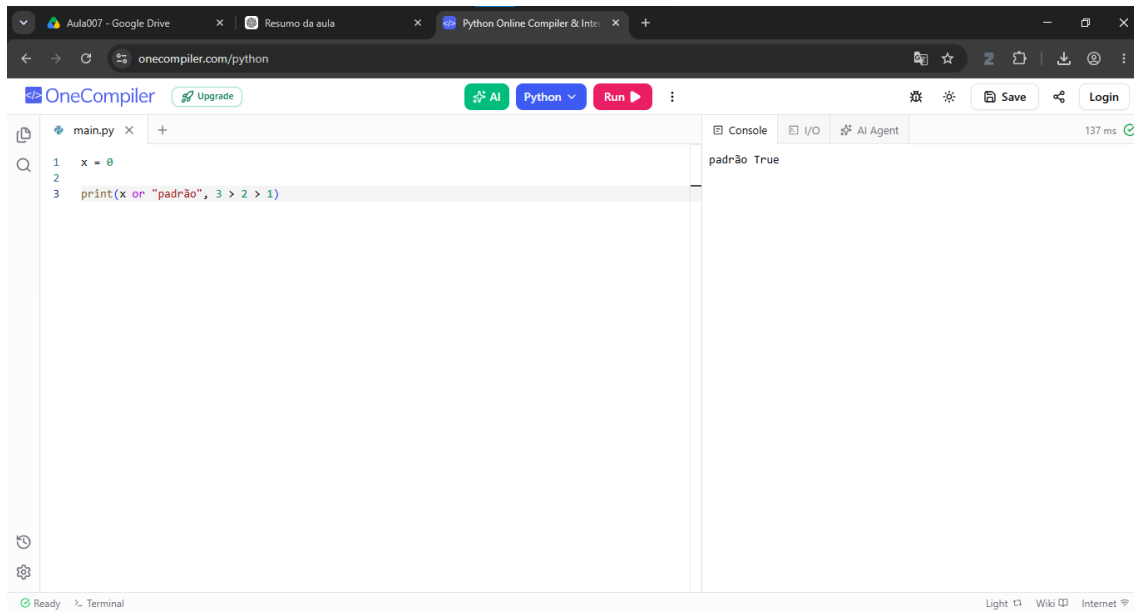
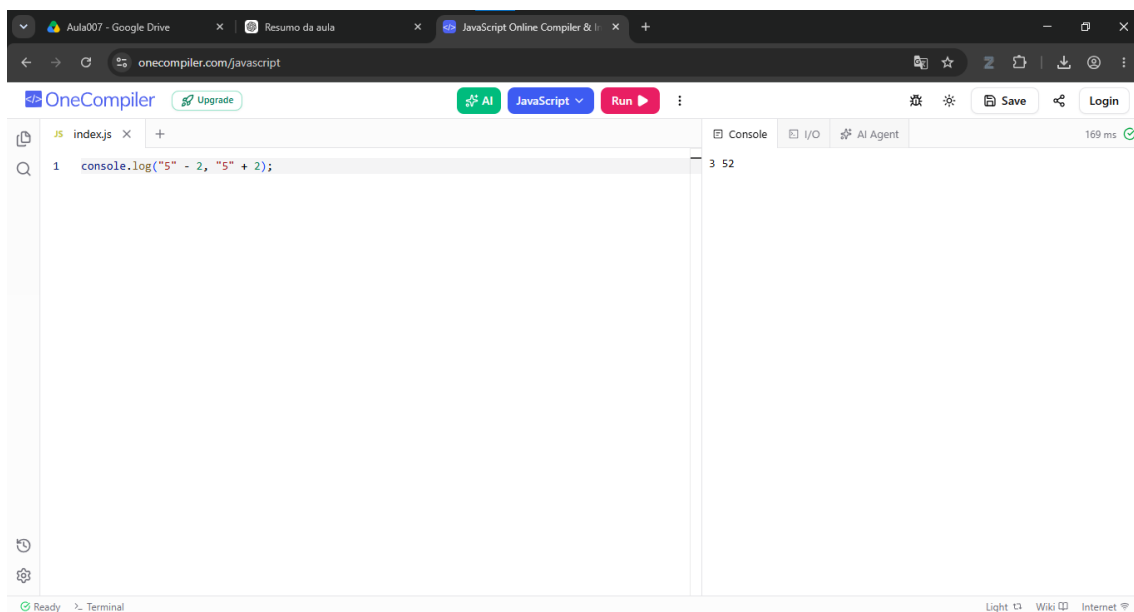


Resumo da Aula 07 – Expressões, Atribuição e Controle



```
1 x = 0
2
3 print(x or "padrão", 3 > 2 > 1)
```

padrão True



```
1 console.log("5" - 2, "5" + 2);
```

3 52

Nas prints acima, a primeira (python) mostra uma comparação entre $x = 0$ e “padrão”. Em python, o OR busca um valor verdadeiro. Como 0 é considerado um valor falso, a linguagem compreende que “padrão” é um valor verdadeiro e apresenta ele. Também é feita a comparação $3 > 2 > 1$, onde é feita a interpretação: $3 > 2$ and $2 > 1$. Como ambas são verdadeiras, é retornado o valor true. Na segunda (JavaScript) mostra uma particularidade da linguagem, onde ela faz cálculos com strings. No primeiro caso, é feito realmente um cálculo, pois na linguagem o operador “-” serve como subtração, enquanto o operador “+” pode servir como soma e concatenador, fazendo com que no segundo caso seja feita a concatenação do número com a string.

Expressões

Expressões são formadas por operadores, operandos, parênteses e chamadas de funções. Para entender o resultado de uma expressão, é importante conhecer as regras de precedência e associatividade dos operadores.

A precedência determina qual operação será realizada primeiro. Por exemplo, em $3 + 4 * 5$, a multiplicação acontece antes da soma, resultando em 23. Já a associatividade determina a ordem quando existem operadores com a mesma precedência.

Os parênteses podem ser usados para deixar a intenção do programador mais clara e evitar interpretações diferentes.

Também foi mostrado que diferentes linguagens podem tratar as mesmas expressões de maneiras diferentes. Python, Java, JavaScript e C possuem regras próprias para operações, conversões e comparações.

Efeitos colaterais e ordem de avaliação

A ordem em que os operandos de uma expressão são avaliados pode ser importante quando uma função modifica uma variável.

Ordem dos operandos e efeitos colaterais

Exemplo do livro (p. 307)

```
int a = 5;

int fun1(void) {
    a = 17;          /* efeito colateral */
    return 3;
}

int main(void) {
    a = a + fun1();  /* 8 ou 20? */
    printf("a = %d\n", a);
}
```

8

Java

a ordem é garantida: da esquerda para a direita (p. 308)

20

C (gcc 14)

a ordem não é definida pelo padrão

20

Go

a especificação não define a leitura de a em relação à chamada

Transparência referencial (p. 308): sem efeitos colaterais, a ordem deixa de importar, e $f(x) + f(x)$ pode virar $2 * f(x)$.

Sebesta, 11ª ed. - §7.2.2 - p. 307-309 - exemplos/02_efeitos_colaterais

Foi apresentado acima o exemplo de uma variável que é alterada dentro de uma função enquanto também é utilizada na mesma expressão. Dependendo da linguagem, o resultado pode ser diferente. Java, por exemplo, garante a avaliação da esquerda para a direita, enquanto o padrão da linguagem C não define essa ordem nesse caso.

Por isso, efeitos colaterais podem tornar o código mais difícil de entender e gerar resultados inesperados.

Sobrecarga de operadores

Operadores sobrecarregados

- O mesmo símbolo com **vários significados**: + soma inteiros, reais e concatena (Java)
- Risco (p. 309): & em C++ é E bit a bit e "endereço de"
- **Definida pelo usuário** (p. 404): C++, C#, Python, Rust, Kotlin. Java **não** permite

Java

```
System.out.println('a' + 1);    // 98
System.out.println("" + 'a' + 1); // a1
x.add(y).multiply(tres);    // BigDecimal: sem +
```

Python - sobrecarga com métodos especiais

```
class Vetor:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __add__(self, outro):
        # v1 + v2
        return Vetor(self.x + outro.x,
                      self.y + outro.y)

    def __mul__(self, k):
        # v * 3
        return Vetor(self.x * k, self.y * k)

v = Vetor(1, 2) + Vetor(3, 4)    # Vetor(4, 6)
print(v * 3)                    # Vetor(12, 18)
```

Sebesta, 11ª ed. - §7.3 - p. 309-310 e §9.11 - p. 404 - exemplos/03_sobrecarga

A sobrecarga de operadores acontece quando um mesmo operador possui diferentes significados dependendo dos tipos envolvidos.

Um exemplo é o operador +, que pode realizar uma soma ou uma concatenação. Em Java, por exemplo, 'a' + 1 resulta em 98, enquanto "" + 'a' + 1 resulta em "a1".

Algumas linguagens também permitem que o próprio programador defina comportamentos para operadores. Java não permite essa sobrecarga definida pelo usuário.

Conversões de tipos

Conversões de tipos: alargamento, estreitamento e coerção

Java · coerção e cast

```
int grande = 123456789;
float f = grande;      // alargamento implícito
System.out.println((int) f); // 123456792 !

double d = 3.99;
System.out.println((int) d); // 3 (cast)
System.out.println((byte) 200); // -56

byte b = 10, c = 20;
// byte s = b + c; // erro: b + c é int (p. 312)

System.out.println(1 / 2); // 0
System.out.println(1 / 2.0); // 0.5
```

Linguagem	int → real (implícito)	int32 + int64
C	sim	coerção
Java	sim (alargamento)	coerção para long
Python	sim	(um só tipo int)
Go	não: float64(i)	erro de compilação
Rust	não: i as f64	erro de compilação
ML, F# (p. 312)	não	erro de compilação

Coerção esconde erros (p. 312): em `d = b * a`; com `a` digitado por engano no lugar de `c`, o Java converte e compila.

Sebesta, 11ª ed. - §7.4 - p. 310-313 - exemplos/04_conversoes

Acima é apresentado as conversões entre tipos, principalmente:

- Alargamento: conversão para um tipo capaz de representar uma faixa maior de valores.
- Estreitamento: conversão para um tipo que pode não comportar todos os valores originais.
- Coerção: conversão automática feita pela linguagem.
- Cast: conversão explícita solicitada pelo programador.

A coerção pode facilitar a escrita do código, mas também pode esconder erros. JavaScript e PHP, por exemplo, realizam várias conversões automáticas.

Em JavaScript, `"5" - 2` resulta em 3, enquanto `"5" + 2` resulta em "52". Foi possível notar também que o operador `==` pode realizar conversões, enquanto `===` compara sem realizar essa coerção.

No PHP, pode ser notado o problema conhecido como type juggling, mostrando como comparações com `==` podem produzir resultados inesperados, conforme mostrado na imagem abaixo.

Coerção implícita: JavaScript e PHP

JavaScript

```
"5" - 2           // 3
"5" + 2           // "52"
"5" * "2"         // 10
true + 1          // 2
[] + {}           // "[object Object]"
null + 1          // 1
undefined + 1     // NaN
"b" + "a" + +"a" + "a" // "baNaN"
```



```
0 == ""          // true
"" == "0"        // false
0 == "0"         // true  (não é transitivo!)
0 === "0"        // false (=== não converte)
```

PHP - "type juggling"

```
var_dump("0e123" == "0e456"); // true !
var_dump("0e123" === "0e456"); // false
var_dump("1" == "01");        // true

md5("240610708") // "0e462097431906..."
md5("QNKCDZO")   // "0e830400451993..."
// == entre os dois hashes: true
```

Regra prática

Em JavaScript e PHP, compare com `===` e `!==`, que **não** fazem coerção.

Sebesta, 11ª ed. - §7.4.1 Coerção em expressões - p. 311-312 - exemplos/04_conversoes

Expressões booleanas e curto-circuito

As expressões relacionais e booleanas são utilizadas principalmente para tomar decisões no programa.

Expressões relacionais e booleanas

C - booleano é inteiro (p. 315)

```
int a = 3, b = 2, c = 1;
printf("%d\n", a > b > c); // 0 !
// (3 > 2) vale 1, e 1 > 1 é falso

printf("%d\n", (a > b) && (b > c)); // 1

if (-1) printf("verdadeiro\n"); // != 0
```

Python - comparações encadeadas

```
a, b, c = 3, 2, 1
print(a > b > c) # True: (a > b) and (b > c)

# contam como FALSO em um if:
# 0 0.0 "" [] {} None
# contam como VERDADEIRO:
# "0" [0] " "
```

Linguagem	O que o <code>if</code> aceita
C, Python, JavaScript	qualquer valor (0, vazio e nulo contam como falso)
Java, C#, Go, Rust	somente booleano : <code>if (n)</code> com <code>n</code> inteiro nem compila

Sebesta, 11ª ed. - §7.5 - p. 313-315 - exemplos/05_booleanas

Um exemplo importante é a diferença entre C e Python. Em C, `3 > 2 > 1` resulta em falso porque a expressão é avaliada da esquerda para a direita. Em Python, essa mesma expressão é uma comparação encadeada e resulta em verdadeiro.

Avaliação em curto-circuito

Java · && faz curto-circuito, & não

```
// busca do livro (p. 316)
while (i < lista.length && lista[i] != chave)
    i++;           // && evita lista[lista.length]

String s = null;
s != null && s.length() > 0 // false
s != null & s.length() > 0 // NullPointerException
```

JavaScript · ?. e ??

```
const u = { nome: "Ana", endereco: null };
u.endereco?.cidade // undefined, sem erro

const qtd = 0;
qtd || 10           // 10 (o 0 se perdeu!)
qtd ?? 10           // 0 (só troca null/undefined)
```

Cuidado com efeitos colaterais (p. 316)

Em `(a > b) || (b++ / 3)`, o `b++` só acontece quando `a <= b`. Em Python, `0 or "padrão"` devolve "padrão": and e or devolvem um dos operandos.

Sebesta, 11ª ed. - §7.6 - p. 315-317 - exemplos/05_booleans

Acima é apresentado o conceito de curto-circuito. Ele ocorre quando a linguagem consegue determinar o resultado de uma expressão lógica sem avaliar todos os seus operandos.

Por exemplo, em:

```
a != null && a.length() > 0
```

se `a` for `null`, a segunda condição não precisa ser avaliada. Isso pode evitar erros durante a execução.

Atribuição

A atribuição é utilizada para colocar valores em variáveis. Algumas linguagens permitem que a atribuição também seja utilizada como uma expressão.

Em C, por exemplo:

```
if (x = y)
```

realiza uma atribuição, e não uma comparação. Esse tipo de situação pode causar erros difíceis de perceber.

Sentenças de atribuição

C · a atribuição é uma expressão (p. 319-320)

```
sum = count = 0;           // múltiplos alvos
a = b + (c = d / b) - 1;    // exemplo do livro

while ((ch = getchar()) != EOF) { ... }

if (x = y) { ... } // compila! atribui e testa y
```

Python · atribuição múltipla (p. 320)

```
a, b = b, a                # troca sem temporária
if (n := len(dados)) > 5: ... # morsa (Python 3.8+)
```

Linguagem	Atribuição é expressão?
C, C++, JavaScript	sim: <code>if (x = y)</code> compila
Java, C#	sim, mas o <code>if</code> exige booleano
Python	não, exceto o operador morsa <code>:=</code>
Go	não: nem <code>x++</code> é expressão
Rust	vale <code>()</code> : <code>if x = 5</code> não compila

Linguagens funcionais (p. 321)

`val` (ML) e `let` (F#, Rust) dão **nome** a um valor, que não muda.

Sebesta, 11ª ed. - §7.7 - p. 317-321 - exemplos/06_atribuicao

Acima são apresentados diferentes tipos de atribuição, como atribuições compostas (`+=`), incrementos (`++`) e atribuições múltiplas, disponíveis em algumas linguagens.

É possível notar ainda que Python, Go e Rust possuem restrições que evitam alguns desses problemas.

Estruturas de controle

As estruturas de controle determinam a ordem em que as instruções são executadas.

Estruturas de controle: quantas são necessárias?

Sequência

uma sentença após a outra

```
a = 1;
b = a + 2;
```

Seleção

escolher um caminho

```
if (x > 0) ...
else ...
```

Iteração

repetir enquanto uma condição vale

```
while (x > 0)
  x--;
```

Böhm e Jacopini (1966): sequência, seleção e laço lógico bastam para qualquer algoritmo. O goto não é necessário.

Por que então tantas outras (`for`, `switch`, `do`, `foreach`)? **Facilidade de escrita.** O limite é a **legibilidade**: quanto maior a linguagem, menos do código alheio você reconhece (p. 329).

Sebesta, 11ª ed. - §8.1 - p. 328-330

No slide mostra também os conceitos de sequência, seleção e iteração. Onde ele nos explica da seguinte forma: Sequência: execução das instruções uma

após a outra. Seleção: escolha entre diferentes caminhos, como if e else.
Iteração: repetição de instruções enquanto uma condição for atendida.

Ele também cita sobre o trabalho de Böhm e Jacopini, que mostrou que sequência, seleção e repetição são suficientes para representar algoritmos.

Estruturas de seleção

Seleção de dois caminhos: o "else pendente"

C: a endentação engana (p. 331)

```
int sum = 0, count = 5, result = -1;
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1;           // de qual if é este else?
printf("%d\n", result);  // 1
```

Regra de Java, C, C++ e C#

O else pertence ao if **mais próximo** que ainda não tem else. A endentação é ignorada (exceto em Python e F#).

Chaves obrigatórias eliminam o problema: Perl, Go e Rust (p. 332).

Seletores que devolvem valor

```
let y = if x > 0 { x } else { 2 * x };
y = x if x > 0 else 2 * x
```

// Rust: if é uma expressão (como em F#, p. 334)
Python: expressão condicional (p. 306)

Sebesta, 11ª ed. - 58.2.1 - p. 330-334 - exemplos/07_selecao

Na imagem acima é apresentada a situação do else pendente. Em linguagens como C, Java, C++ e C#, o else pertence ao if mais próximo que ainda não possui um else. Neste caso, a endentação pode dar uma impressão diferente da interpretada pelo compilador. Para evitar este tipo de problema, é recomendado o uso de chaves

Seleção múltipla: do switch de C ao match

C · sem break, o controle "cai" (p. 335)

```
switch (index) {
  case 1:
  case 3: odd += 1;
  case 2:
  case 4: even += 1;
  default: printf("Error\n");
}
// index = 1: odd, even E erro!
```

Java 14+ · switch como expressão

```
String tipo = switch (dia) {
  case SAB, DOM -> "fim de semana";
  case SEG, TER, QUA,
    QUI, SEX -> "dia útil";
}; // sem break, sem cair;
// exaustivo para enum
```

Rust · match exaustivo

```
match n {
  i32::MIN..=-1 => "negativo",
  0 => "zero",
  1..=9 => "um dígito",
  _ => "vários dígitos",
}
// sem o _ : não compila
```

Go: cada caso termina sozinho, e cair exige fallthrough. C#: cair implicitamente é proibido (p. 336). Python 3.10: match com padrões e guardas.

Sebesta, 11ª ed. - §8.2.2 - p. 334-341 - exemplos/07_selecao

Na imagem acima é apresentado um exemplo de switch, usado para selecionar entre vários casos. Em C e em algumas versões de outras linguagens, esquecer o break pode fazer a execução continuar nos casos seguintes.

Já o Java possui uma forma mais recente de switch, utilizando ->, que evita esse comportamento. Rust utiliza match, que exige que os casos sejam tratados de forma exaustiva.

Laços de repetição

Os laços permitem executar um conjunto de instruções várias vezes.

Laços controlados por contador

C · o for é um laço lógico disfarçado

```
for (int i = 0; i < 10; i++) {
  printf("%d ", i);
  i += 2;           // altera a contagem
}                  // 0 3 6 9

for (; i <= 100; ) // expressões opcionais
  soma += i++;
```

Python · o for percorre um iterável

```
for i in range(0, 10):
  print(i, end=" ")
  i += 2           # NÃO muda nada
                  # 0 1 2 3 4 5 6 7 8 9

for i in range(10, 0, -3): # 10 7 4 1
  ...
```

Questões de projeto (p. 342)

Qual é o tipo e o escopo da variável? Posso alterá-la no corpo? Os parâmetros são avaliados uma vez ou a cada volta?

Sebesta, 11ª ed. - §8.3.1 - p. 342-345 - exemplos/08_iteracao

O for pode funcionar de maneiras diferentes dependendo da linguagem. Em C e Java, ele normalmente é utilizado com uma variável de controle. Em Python, o for percorre um objeto iterável.

Na imagem é mostrado que alterar a variável do laço dentro do corpo pode produzir resultados diferentes. Em C, essa alteração pode modificar a contagem, enquanto em Python alterar a variável dentro do for não modifica os valores que serão fornecidos pelo range.

Zune (2008) - o laço lógico sem progresso

```
while (dias > 365) {  
    if (bissexto(ano)) {  
        if (dias > 366) { dias -= 366; ano++; }  
        // dias == 366: nada muda, laço infinito!  
    } else { dias -= 365; ano++; }  
}
```

Pós-teste (do-while) executa **ao menos uma vez**, e isso é fácil de esquecer (p. 347).

Sebesta, 11ª ed. - §8.3.2-8.3.4 - p. 345-352 - exemplos/08_iteracao

Um exemplo importante foi o problema dos tocadores Zune, em 2008, causado por um laço que não fazia progresso em uma determinada situação. Como a condição continuava verdadeira e nenhuma variável era modificada, o laço não terminava.

Goto

O goto permite realizar um desvio direto para outro ponto do programa. Apesar de ser uma estrutura poderosa, seu uso excessivo pode tornar o código difícil de compreender e manter.

Desvio incondicional: o goto

"goto fail" (Apple, 2014) - versão simplificada

```
int valida_certificado(void) {  
    int err;  
    if ((err = verifica_hash()) != 0)  
        goto fail;  
    goto fail; // duplicada: desvia SEMPRE  
    if ((err = verifica_assinatura()) != 0)  
        goto fail; // nunca é alcançada  
fail:  
    return err; // 0 = "certificado válido"  
}
```

Dijkstra (1968)

"Go To Statement Considered Harmful"

Quem tem goto hoje

C, C++, C#, Go. Java **não** tem (p. 353)

Gotos restritos

break, continue e return melhoram a legibilidade (p. 353)

Com chaves obrigatórias (Go, Rust) ou com gcc -Wall (aviso de endentação enganosa), o bug teria sido pego.

Sebesta, 11ª ed. - §8.4 - p. 352-353 - exemplos/09_desvio_protegido

A imagem apresenta o caso conhecido como "goto fail", ocorrido na Apple em 2014. Uma linha goto fail duplicada, associada a um if sem chaves, fazia com que uma verificação importante fosse ignorada.

O exemplo mostrou a importância de utilizar estruturas claras, chaves, avisos do compilador, revisão de código e testes.

Comandos protegidos

Comandos protegidos de Dijkstra

```
if x >= y -> max := x
[] y >= x -> max := y
fi
```

Exemplo do livro (p. 354): se $x == y$, qualquer ramo serve.

- Todas as **guardas** são avaliadas
- Várias verdadeiras: escolha **não determinística**
- Nenhuma verdadeira: erro (no `if`) ou fim do laço (no `do ... od`)
- Base do **CSP** de Hoare (1978) e do `select` de Go

Go · `select`, herdeiro dos comandos protegidos

```
a <- "a"      // os dois canais
b <- "b"      // estão prontos

select {      // Go sorteia uma guarda
case x := <-a:
    contagem[x]++
case x := <-b:
    contagem[x]++
}

// 1000 repetições:
// map[a:505 b:495] (muda a cada execução)
```

Sebesta, 11ª ed. - §8.5 - p. 353-355 - exemplos/09_desvio_protegido

Na imagem acima é mostrado os comandos protegidos de Dijkstra. Eles utilizam condições chamadas de guardas. Quando mais de uma guarda é verdadeira, uma delas pode ser escolhida de forma não determinística.

Esse conceito está relacionado ao CSP de Hoare e também ao `select` da linguagem Go, em que diferentes operações disponíveis podem ser escolhidas.